Основы обработки естественного языка

Нормализация, токенизация, типовые задачи и RNN-модели

Курс «Deep Learning»

2026

Содержание

1. Что такое обработка естественного языка?
2. Токенизация и нормализация текста
3. Типовые задачи NLP
4. Рекуррентные нейронные сети в NLP
5. Архитектуры LSTM, GRU и Seq2Seq

Что такое обработка естественного языка?

Определение

NLP — область на стыке ИИ, лингвистики и информатики, изучающая методы взаимодействия компьютера и человеческого языка.

История: зародилась в 1950-х (тест Тьюринга). Правила и статистика сменились DL: word2vec, seq2seq, Transformer.

Конвейер NLP: Предобработка → Векторизация → Модель

Области применения

- Чат-боты и голосовые помощники
- Поиск и ранжирование
- Машинный перевод
- Анализ тональности
- Аннотация текстов и NER
- Генерация текстов

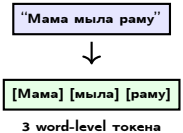
Токенизация

Что такое токенизация?

Процесс разбиения текста на минимальные осмысленные единицы — **токены**: слова, подслова или символы.

Обязательный первый шаг NLP-конвейера. От качества токенизации зависит результат всех последующих этапов.

Пример:



Основные подходы:

Word-level

Простое разделение по пробелам. Большой словарь (10^5–10^6). Проблема OOV (out-of-vocabulary).

Subword (BPE, WordPiece)

Баланс размера словаря и покрытия. Решает OOV. Стандарт: BPE в GPT, WordPiece в BERT.

Character-level

Минимальный словарь (алфавит). Но длинные последовательности, сложнее улавливать смысл.

Нормализация текста

1. Нижний регистр
«Москва» и «москва» — один токен.
Уменьшает словарь.

2. Удаление стоп-слов
Предлоги, союзы, местоимения несут мало смысла и удаляются (для некоторых задач).

3. Очистка от спецсимволов
Удаление пунктуации, URL, HTML-тегов, цифрового мусора.

4. Стемминг
Механическое отсечение окончаний. Быстро, но грубо.
«бегущий» $\rightarrow$ *«бегущ»*, *«красивее»* $\rightarrow$ *«красив»*
Алгоритмы: Портера, Snowball.

5. Лемматизация
Лингвистически корректное приведение к начальной форме (лемме) через словарь и морфологию.
«бежал» $\rightarrow$ *«бежать»*, *«красивее»* $\rightarrow$ *«красивый»*
Инструменты: NLTK, spaCy, pymorphy2 (русский).

Типовые задачи NLP

01 Классификация

Определение категории текста: спам, тематика, жанр.

02 Тональность

Эмоциональная окраска текста: позитив/негатив/нейтрально.

03 Машинный перевод

Автоматический перевод между языками.

04 NER

Named Entity Recognition: извлечение имён, дат, организаций.

05 Генерация

Создание текста: саммаризация, диалоговые системы.

06 QA

Question Answering: поиск ответа в заданном контексте.

07 Language Modeling

Предсказание следующего слова. Основа GPT.

08 Парсинг

Синтаксический анализ, dependency parsing.

09 Textual Entailment

Определение логической связи между текстами.

Рекуррентные нейронные сети в NLP

Принцип работы RNN

Обрабатывают элементы последовательности по одному, сохраняя скрытое состояние (память), которое передаётся на каждом шаге.

Почему RNN для языка? Язык — последовательная структура. RNN учитывают порядок слов и могут моделировать контекст.

Ключевая формула:

$$h_t = f(Wh_{t-1} + Ux_t + b)$$

Проблемы RNN

Затухание и взрыв градиентов: при длинных последовательностях базовая RNN теряет информацию о ранних элементах.

LSTM (Hochreiter & Schmidhuber, 1997):

- Cell state как «конвейер» информации
- Forget gate: что забыть
- Input gate: что добавить
- Output gate: что вывести

GRU: упрощённая LSTM с двумя gate (reset, update). Быстрее, меньше памяти, сопоставимое качество.

LSTM vs GRU

LSTM

- Три gate: forget, input, output
- Отдельная ячейка состояния (cell state)
- Больше параметров
- Лучше на длинных последовательностях
- Доминировал до появления Transformer

$$
\begin{aligned}
f_t &= \sigma(W_f[h_{t-1}, x_t] + b_f) \\
i_t &= \sigma(W_i[h_{t-1}, x_t] + b_i) \\
\tilde{C}_t &= \tanh(W_C[h_{t-1}, x_t] + b_C) \\
C_t &= f_t \odot C_{t-1} + i_t \odot \tilde{C}_t \\
o_t &= \sigma(W_o[h_{t-1}, x_t] + b_o) \\
h_t &= o_t \odot \tanh(C_t)
\end{aligned}
$$

GRU

- Два gate: reset, update
- Единое скрытое состояние (нет cell state)
- Меньше параметров
- Сопоставимое качество на коротких текстах
- Предпочтителен при ограниченных ресурсах

$$
\begin{aligned}
r_t &= \sigma(W_r[h_{t-1}, x_t] + b_r) \\
z_t &= \sigma(W_z[h_{t-1}, x_t] + b_z) \\
\tilde{h}_t &= \tanh(W[r_t \odot h_{t-1}, x_t] + b) \\
h_t &= (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t
\end{aligned}
$$

Seq2Seq и механизм внимания

Encoder-Decoder

Энкодер сжимает входную последовательность в вектор контекста, декодер генерирует выход. Используется в переводе и саммаризации.

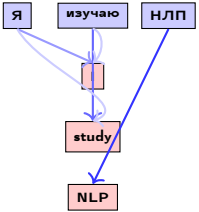
Проблема: при длинных предложениях информация теряется при «упаковке» в один вектор.

Решение — Attention:

$$c_t = \sum_{j=1}^{T} \alpha_{tj} h_j$$

$$\alpha_{tj} = \frac{\exp(e_{tj})}{\sum_k \exp(e_{tk})}$$

$$e_{tj} = a(s_{t-1}, h_j)$$



Seq2Seq + Attention Толщина линии = вес внимания

Фундамент для Transformer: внимание стало основой Transformer (Vaswani et al., 2017) — архитектуры GPT, BERT и всех современных LLM.

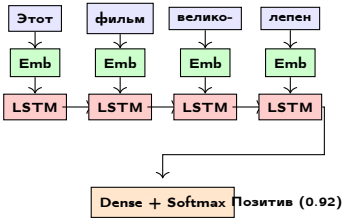
RNN в задаче классификации текста

Анализ тональности

RNN читает текст слово за словом, накапливая контекст. Последнее скрытое состояние содержит суть всего текста и подаётся на классификатор.

BiLSTM-подход: Двунаправленный LSTM обрабатывает текст слева направо и справа налево, захватывая полный контекст.

Pooling состояний: Max-pooling по всем шагам — альтернатива последнему состоянию.



Пример: отзыв «Этот фильм великолепен!» $\rightarrow$ 4 шага LSTM $\rightarrow$ P(позитив) $= 0.92$

RNN в NER (BiLSTM-CRF)

Архитектура BiLSTM-CRF

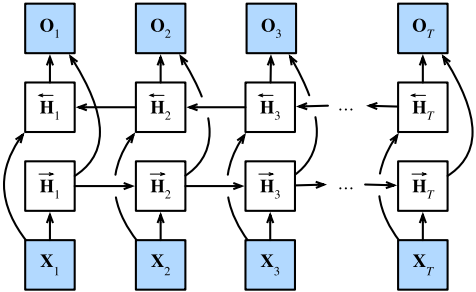
BiLSTM извлекает контекстные признаки для каждого токена. CRF-слой обеспечивает согласованность меток (B-PER не может идти после I-ORG).

Почему двунаправленный? Для NER нужен контекст с обеих сторон: «Маша работает в Яндекс».

BIO-тегирование:

- **B** — начало сущности
- **I** — продолжение сущности
- **O** — не сущность
- PER — персона, ORG — организация, LOC — локация

Пример: «Иван работает в Газпром с 2010 года»

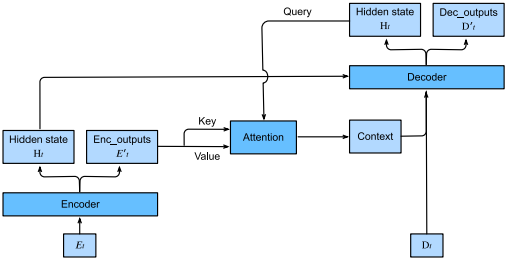


RNN в машинном переводе (Seq2Seq)

Encoder-Decoder

Энкодер сжимает исходное предложение в последовательность скрытых состояний. Декодер генерирует перевод, используя механизм внимания.

Внимание: на каждом шаге декодер вычисляет веса для всех состояний энкодера. Взвешенная сумма фокусируется на релевантных частях.
Пример: при переводе «Я изучаю нейронные сети» на шаге «study» внимание сфокусировано на «изучаю».

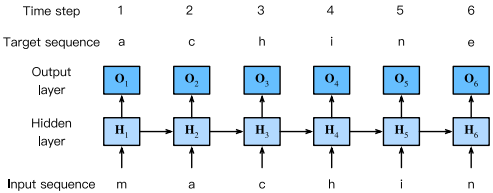


RNN в языковом моделировании и генерации

Языковая модель

RNN предсказывает следующее слово. Обучается на корпусах, максимизируя вероятность правильного продолжения:

$$P(w_1, \ldots, w_T) = \prod P(w_t | w_{<t})$$



Авторегрессия: на генерации модель подаёт собственный выход как следующий вход.

Управление разнообразием:

- Temperature τ — размывает распределение
- Top-K: семплинг из K лучших
- Top-P (nucleus): семплинг по кум. массе

Сравнение подходов в NLP

Критерий	Базовая RNN	LSTM / GRU	Transformer
Длинные зависимости	Слабо	Хорошо	**Отлично**
Параллелизация	Нет	Нет	**Да**
Скорость обучения	Быстрая	Средняя	Высокая (GPU)
Качество	Низкое	Среднее	**SOTA**
Потребление памяти	Мало	Средне	Много
Типичное применение	Простые задачи	Перевод, NER	LLM, чат-боты

Вывод

RNN/LSTM остаются актуальны для задач с ограниченными ресурсами, streaming-данных и real-time обработки. Transformer доминирует в качестве, но требует больше вычислительных ресурсов.

Заключение

Что мы узнали:

- NLP-конвейер: токенизация $\rightarrow$ нормализация $\rightarrow$ модель
- Word, subword (BPE/WordPiece) и character-токенизация
- Стемминг vs лемматизация
- RNN, LSTM, GRU для последовательностей
- Seq2Seq + Attention для перевода

Применения RNN в NLP:

- Классификация и тональность (BiLSTM)
- NER (BiLSTM-CRF)
- Машинный перевод (Seq2Seq + Attention)
- Языковое моделирование
- Генерация текста (авторегрессия)

Дальше: Transformer (лекция 06)